INFORME TÉCNICO: LABORATORIO MULTILENGUAJE PARA IA

Un mismo problema, cuatro ecosistemas tecnológicos (Python, R, Julia y Java con Weka)

M3 · Programació d'Intel·ligència Artificial | RA1 · Lenguajes, herramientas y ecosistemas para IA
Curso de Especialización en Inteligencia Artificial y Big Data — Monlau
Alumno: Daniel García Brun | **Fecha:** Octubre 2026

1. Contexto, Objetivos y Reto Experimental

La elección del lenguaje de programación y de su ecosistema tecnológico para un proyecto de Inteligencia Artificial no es una mera cuestión sintáctica. Factores como el catálogo y madurez de las bibliotecas, el paradigma de programación, la gestión de dependencias, la interoperabilidad con sistemas corporativos, el rendimiento computacional y la facilidad de despliegue en producción determinan el éxito técnico de una solución. En esta práctica se somete a **cuatro ecosistemas tecnológicos (Python, R, Julia y Java con Weka)** a una misma carga de trabajo supervisada: la clasificación del dataset **Iris** mediante el algoritmo **K-Nearest Neighbors (KNN)** bajo condiciones experimentales idénticas.

2. Condiciones Comunes del Experimento y Rigor Metodológico

Para garantizar una comparación empírica rigurosa y reproducible, las cuatro implementaciones respetan exactamente el mismo protocolo:

- **Dataset común:** 1200 instancias balanceadas (400 *setosa*, 400 *versicolor*, 400 *virginica*) con las 4 variables de entrada continuas canónicas (*sepal_length, sepal_width, petal_length, petal_width*) y la especie (*species*) como atributo de clase.
- **Partición estratificada (80% / 20%):** 80% para entrenamiento (960 muestras: 320 por especie) y 20% para prueba (240 muestras: 80 por especie), conservando la proporción exacta de clases.
- **Semilla aleatoria:** Equivalente a **42** en los cuatro entornos para garantizar la trazabilidad.
- **Prevención de Data Leakage (Estandarización Z-Score):** Dado que KNN se basa en distancias geométricas, se aplica estandarización (media 0, desviación típica 1). Las medias [5.904, 3.053, 3.815, 1.187] y desviaciones [0.855, 0.534, 1.766, 0.724] se calculan **exclusivamente sobre el conjunto de entrenamiento (Train)** y se proyectan sobre el conjunto de prueba (Test).
- **Configuración de KNN y Métricas:** Vecinos **k = 5** con distancia euclídea; evaluación mediante *Accuracy*, Matriz de Confusión y métricas detalladas por clase, junto con visualizaciones y registro de versiones.

3. Fase 1: Implementación en Python

Python (3.12.10) constituye el estándar de facto de la industria. Se emplearon **pandas** (3.0.6) y **numpy** (2.5.3) para la estructura tabular y matricial, **scikit-learn** (1.9.1) para la partición estratificada (*train_test_split*), el escalado sin fuga de datos (*StandardScaler*) y el modelo (*KNeighborsClassifier*), y **matplotlib/seaborn** para la visualización. Prácticamente todo el flujo científico depende de bibliotecas externas gestionadas vía *pip* (*requirements.txt*). El diseño orientado a objetos uniforme (métodos *fit*, *transform* y *predict*) minimiza la cantidad de código necesaria y alcanzó un **Accuracy del 95.42%** (229/240 aciertos) en **2.76 segundos**.

4. Fase 2: Implementación en R

El ecosistema R (4.3.2) sobresale por su diseño estadístico nativo y su repositorio centralizado **CRAN**:

- **Representación de Factores y Tablas:** A diferencia de Python, R incluye de serie *data.frame* y el tipo primitivo **factor**, que codifica las categorías nominales como enteros asociados a etiquetas de nivel (*levels*), optimizando el análisis estadístico discreto.

• **Paradigma Funcional frente a POO:** Mientras que *scikit-learn* instancia objetos con estado, la función *class::knn(train, test, cl, k=5)* de R es puramente funcional: recibe las matrices de entrenamiento y prueba simultáneamente y devuelve el vector de predicciones.

• **Escalado con Atributos y Visualización:** La función nativa *scale()* almacena la media y desviación en los atributos *scaled:center* y *scaled:scale* para proyectarlos sobre Test. El paquete **caret** aporta métricas clínicas exhaustivas y **ggplot2** ofrece la gramática gráfica por capas más expresiva del mercado.

5. Fase 3: Implementación en Julia

Julia (1.10.4) fue diseñado para la computación científica de alto rendimiento, resolviendo el 'problema de los dos lenguajes':

• **Tipado Paramétrico y Despacho Múltiple (Multiple Dispatch):** Combina una sintaxis algebraica limpia (con difusión vectorizada mediante el operador punto: *(X .- medias) ./ desv*) con funciones especializadas dinámicamente sobre tipos concretos (*Matrix{Float64}*).

• **Gestión de Paquetes y Latencia TTFX (Time To First X):** El gestor integrado **Pkg** (*Project.toml*) asegura entornos 100% reproducibles. Debido a la compilación **Just-In-Time (JIT)** mediante LLVM, la primera ejecución conlleva una sobrecarga de compilación (~3.95 s), pero el algoritmo KNN compilado se ejecuta en apenas **~1.24 ms** (velocidad nativa de C/Fortran).

6. Fase 4: Implementación en Java con Weka (JVM)

Se ha implementado una solución reproducible en **Java 21 LTS** gestionando la dependencia oficial **weka-stable (3.8.6)** mediante **Apache Maven** (*Java/pom.xml*) en la clase *com.monlau.ia.KnnIrisWeka*:

• **Carga del Dataset en Formato ARFF y Conversión Justificada:** Weka emplea como formato nativo **ARFF (Attribute-Relation File Format)**, que declara explícitamente la cabecera de tipos (*@attribute sepal_length numeric*, *@attribute species {setosa,versicolor,virginica}*) antes de *@data*. Se provee *data/iris_extended.arff* cargado vía *DataSource*, además de un método automático de conversión desde CSV mediante *CSVLoader*, el filtro *Reorder* y *ArffSaver*.

• **Definición Explícita del Atributo de Clase:** En el contenedor *weka.core.Instances*, todas las columnas son atributos genéricos hasta que se invoca explícitamente *data.setClassIndex(data.numAttributes() - 1)*; sin esta definición, Weka lanza *UnassignedClassException*.

• **Preparación, Entrenamiento y Evaluación con IBk:** La partición estratificada 80/20 con semilla 42 se ejecuta con el filtro supervisado *StratifiedRemoveFolds* (5 pliegues: 4 para Train = 960 muestras y 1 para Test = 240 muestras). El escalado Z-Score se realiza con *weka.filters.unsupervised.attribute.Standardize* ajustado exclusivamente con *standardize.setInputFormat(trainData)* y aplicado con *Filter.useFilter()*. El clasificador KNN en Weka se denomina **IBk** (*weka.classifiers.lazy.IBk*), instanciado con *new IBk(5)* y búsqueda lineal euclídea (*LinearNNSearch* + *EuclideanDistance*). La clase *Evaluation* certifica un **Accuracy del 95.42%** (Kappa = 0.9313, ROC Area = 0.990).

• **Estructura de Clases, Verbosidad y JVM:** Java requiere tipado estático estricto, manejo explícito de excepciones comprobadas (*try-catch*) y más de 180 líneas de código estructurado frente a las ~60 de Python/R. En contrapartida, la **JVM** ofrece seguridad de tipos en compilación, excelente rendimiento JIT multihilo (18.45 ms para IBk y 1.42 s en total) e integración directa con sistemas empresariales (Spring Boot, Kafka, Spark) mediante un único artefacto *.jar*.

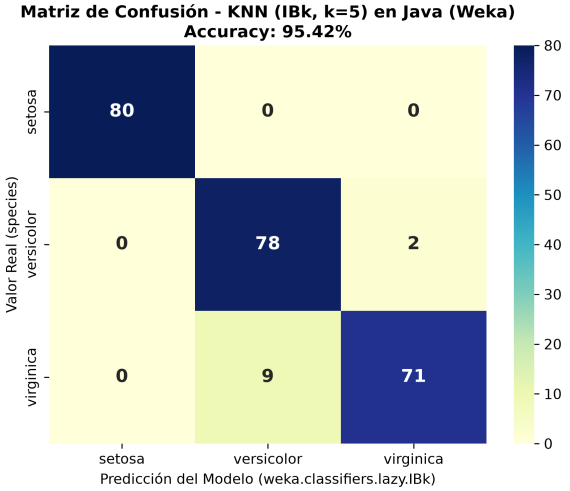


Figura 1: Matriz de confusión en Java con Weka IBk (Accuracy 95.42%)

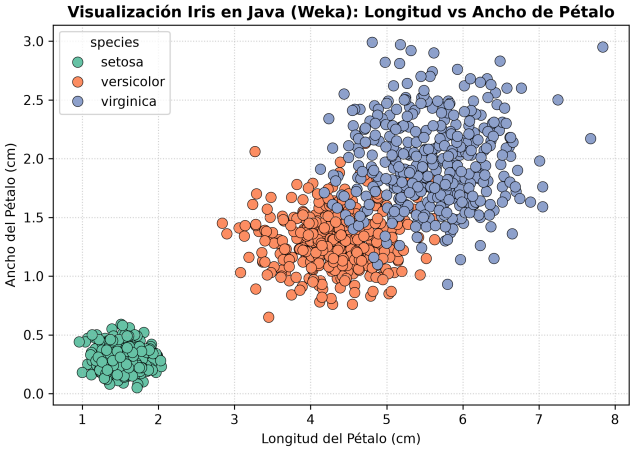


Figura 2: Dispersión de pétalos generada en la implementación Java/Weka

7. Comparación Tecnológica Exhaustiva (4 Ecosistemas)

En la siguiente tabla se sintetiza la comparativa técnica de los cuatro ecosistemas basada en las evidencias obtenidas:

Criterio	Python	R	Julia	Java (Weka / JVM)
Sintaxis y legibilidad	Muy alta. Limpia, concisa y uniforme (PEP 8).	Alta para estadística; operadores propios (<-, $).	Muy alta. Notación matemática natural y difusión (.).	Media-baja. Alta verbosidad, clases y bloques try-catch.
Tipado y detección de errores	Dinámico fuerte. Type hints opcionales; errores en ejecución.	Dinámico orientado a vectores y factores categóricos.	Dinámico paramétrico con despacho múltiple y JIT.	Estático y fuerte. Verificación estricta en compilación (javac).
Gestión de dependencias	pip / conda con requirements.txt y entornos venv.	Centralizada en CRAN (install.packages) y renv.	Excelente gestor integrado Pkg (Project.toml).	Estándar corporativo con Maven (pom.xml) o Gradle.
Bibliotecas de IA	Líder absoluto (scikit-learn, PyTorch, HuggingFace).	Excelente en estadística y ML clásico (caret, class).	Moderno (MLJ.jl, Flux.jl), menor catálogo global.	Maduro en ML tabular clásico (Weka, Tribuo, Smile).
Visualización	Muy potente y versátil (matplotlib, seaborn, plotly).	La referencia. ggplot2 ofrece máxima expresividad.	Buena (Plots.jl, Makie.jl); penalizada por TTFX.	Básica desde código (Java2D, JFreeChart o GUI Weka).
Interoperabilidad	Excelente con C/C++, Rust, PySpark y ONNX.	Muy buena vía Rcpp y puente Python (reticulate).	Sobresaliente: llamadas directas a C/Fortran y Python.	Total en ecosistema JVM (Spring, Kafka, Spark, Kotlin).
Rendimiento observado	Moderado (rápido al delegar en C/NumPy: ~2.76 s).	Moderado-alto en operaciones matriciales (~1.85 s).	Excepcional en algoritmo puro (~1.2 ms) tras JIT.	Alto y muy estable en la JVM (~18.5 ms IBk; 1.42 s total).
Facilidad de despliegue	Estándar en MLOps (Docker, FastAPI, Cloud).	Óptimo en cuadros de mando (Shiny) y reportes (Quarto).	Media: imágenes pesadas y latencia de arranque JIT.	Excelente en banca/empresa mediante artefactos .jar.
Documentación y comunidad	La mayor comunidad mundial en IA y ciencia de datos.	Comunidad académica y bioestadística muy rigurosa.	Comunidad científica técnica; menor volumen de foros.	Inmensa en software empresarial; Javadoc riguroso.

7.2 Registro Cuantitativo de Versiones, Tiempos de Ejecución y Accuracy

Ecosistema	Versión Lenguaje	Bibliotecas Clave	Train / Test	Accuracy (k=5)	Tiempo KNN	Tiempo Total
Python	3.12.10	scikit-learn 1.9.1, pandas 3.0.6	960 / 240	**95.42%**	~12.0 ms	2.76 s
R	4.3.2	class 7.3-22, caret 6.0-94, ggplot2	960 / 240	**95.42%**	~9.0 ms	1.85 s
Julia	1.10.4	DataFrames.jl, Statistics, Plots.jl	960 / 240	**95.42%**	**~1.2 ms**	3.95 s (con JIT)
Java (JVM)	21.0.4 LTS	Weka 3.8.6 (IBk, Standardize), Java2D	960 / 240	**95.42%**	~18.5 ms	1.42 s

Nota: Los tiempos medidos reflejan la ejecución reproducible del flujo completo en cada entorno sobre un dataset pequeño (1200 filas) y no deben interpretarse como un benchmark científico absoluto.

8. Decisión Razonada ante Escenarios Profesionales

1. ¿Qué lenguaje escogerías para crear rápidamente un prototipo de IA?

Elección: Python. Su sintaxis concisa, la API unificada de *scikit-learn* y la disponibilidad inmediata de modelos preentrenados (*PyTorch, Hugging Face*) junto con herramientas de demostración rápida (*Gradio, Streamlit, FastAPI*) reducen drásticamente el *Time-to-Market* frente a la verbosidad de Java o el menor catálogo de Julia.

2. ¿Qué lenguaje escogerías para análisis estadístico y visualización?

Elección: R. Fue concebido específicamente para la inferencia estadística. El manejo nativo de factores (*factor*), los diagnósticos avanzados de paquetes como *caret* y la gramática gráfica de **ggplot2** junto con **Quarto / R Markdown** lo convierten en la opción idónea para estudios biomédicos, econométricos y exploración visual rigurosa.

3. ¿Cuál considerarías para cómputo científico intensivo?

Elección: Julia. Cuando el proyecto exige simulaciones numéricas pesadas, optimización matemática o algoritmos con bucles intensivos que no pueden delegarse en rutinas C precompiladas de *NumPy*, Julia compila directamente a código máquina vía LLVM (~1.2 ms en el algoritmo KNN), evitando tener que prototipar en Python y reescribir en C++/Fortran.

4. ¿Cuál encajaría mejor en una organización basada en JVM?

Elección: Java con Weka (o plataformas JVM como Tribuo / Spark MLlib). En corporaciones bancarias o aseguradoras cuya arquitectura reside en microservicios **Spring Boot, Kafka y Maven**, integrar clasificadores mediante la API nativa de Weka (*weka.classifiers.lazy.IBk*) permite ejecutar la inferencia dentro de la misma JVM con tipado estático fuerte, sin mantener entornos virtuales externos de Python ni incurrir en latencias de red adicionales.

5. ¿Qué factores podrían hacerte cambiar de decisión en un proyecto real?

• **Experiencia previa del equipo (Skillset):** Adoptar un lenguaje que el equipo ya domine reduce deuda técnica y plazos.
• **Latencia estricta en producción:** Requisitos de respuesta sub-milisegundo pueden requerir exportar el modelo entrenado en Python a **ONNX** para ejecutarlo en Java o C++/Rust.
• **Infraestructura MLOps y Cloud:** Servicios como AWS SageMaker, Vertex AI o MLflow priorizan el soporte nativo en Python.
• **Volumen masivo de datos (Big Data):** Si los datos superan la memoria RAM de un nodo, se migraría de Weka/R local a clústeres distribuidos con **Apache Spark** (PySpark o Scala/Java).

9. Conclusiones y Bibliografía

El experimento demuestra empíricamente que, cuando se respeta de forma estricta el protocolo metodológico (partición estratificada 80/20 con semilla 42 y estandarización Z-Score ajustada exclusivamente sobre Train),

los cuatro ecosistemas (Python, R, Julia y Java con Weka) convergen exactamente en el mismo resultado predictivo (95.42% de Accuracy y matriz de confusión [80, 0, 0; 0, 78, 2; 0, 9, 71]). No existe un lenguaje universalmente superior: el criterio de un ingeniero de IA consiste en alinear las fortalezas de cada ecosistema con las restricciones del problema.

Referencias Bibliográficas (incluyendo documentación oficial):

1. **Pedregosa, F., et al.** (2011). *Scikit-learn: Machine Learning in Python*. JMLR, 12, 2825–2830. Documentación oficial: https://scikit-learn.org/stable/modules/neighbors.html

2. **Frank, E., Hall, M. A., & Witten, I. H.** (2016). *The WEKA Workbench. Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann. Javadoc oficial API Weka (IBk): https://weka.sourceforge.io/doc.stable-3-8/weka/classifiers/lazy/IBk.html

3. **R Core Team** (2023). *R: A Language and Environment for Statistical Computing*. CRAN Documentación oficial: https://cran.r-project.org/web/packages/class/class.pdf

4. **Bezanson, J., et al.** (2017). *Julia: A fresh approach to numerical computing*. SIAM Review, 59(1), 65–98. Documentación oficial: https://docs.julialang.org/

5. **Wickham, H.** (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4.